INSTITUTO FEDERAL DE MATO GROSSO DO SUL
CAMPUS NAVIRAÍ

NOTAS DE AULA DE

ANÁLISE E PROJETO ORIENTADO A OBJETO

I

Prof. MSc. Luiz F. Picolo

NAVIRAÍ - MS

Atualizado em 27 de fevereiro de 2020

1 Introdução

Não é o mais forte que sobrevive, nem o mais inteligente.
Quem sobrevive é o mais disposto à mudança.
Charles Darwin

Praticamente todos os países, hoje em dia, dependem de complexos sistemas com base em computadores. Cada vez mais os produtos incorporam, de algum modo, computadores e software de controle. Nesses sistemas, o software representa uma grande e crescente proporção do custo total do sistema. Por isso, produzir software de um modo que apresente uma boa relação custo-benefício é essencial para o funcionamento das economias nacionais e inter- nacionais [Sommerville 2003].

1.1 Engenharia de Software

Ao se citar o nome Engenharia, muitas áreas associadas a ela despontam em nossa mente, tais como engenharia civil, engenharia elétrica, engenharia mecânica, entre tantas outras. Dentre estes vários ramos da engenharia se encontra a **Engenharia de software**. A engenharia de software é uma disciplina da engenharia, cuja meta é o desenvolvimento de sistemas de software com boa relação custo-benefício [Sommerville 2003].

1.1.0.1 Objetivos

Podemos destacar, dentro vários objetivos, quatro. Nestes, a engenharia de software tem como objeto:

- abordagem sistemática e organizada;
- Organização de um processo;
- Reduzir o custo com manutenção (80%);
- Produzir software de alta qualidade.

1.1.1 O que é software?

Muita gente associa o termo software aos programas de computador. Na verdade, essa é uma visão muito restritiva. Software não é apenas o programa mas também toda a documentação associada e os dados de configuração necessários para fazer com que esses pro- gramas operem corretamente [Sommerville 2003]. Em geral, os engenheiros de

software adotam uma abordagem sistemática e organizada em seu trabalho, uma vez que essa é, com frequência, a maneira mais eficaz de produzir software de alta qualidade.

Os engenheiros de software se ocupam de desenvolver produtos de software, isto é, software que possa ser vendido a um cliente. Há dois tipos de produtos de software:

1. **Produtos genéricos:** São sistemas do tipo *stand-alone*, produzidos por uma organização de desenvolvimento e vendidos no mercado a qualquer cliente capaz de adquiri-los.

2. **Produtos sob encomenda:** São os sistemas encomendados por um cliente em particular. O software é desenvolvido especialmente para aquele cliente por uma empresa de software.

1.1.2 Processo de software

Segundo Sommerville 2003 um processo de software é um conjunto de atividades e resultados associados que geram um produto de software. Para o autor há quatro atividades de processo fundamentais:

1. **Levantamento de requisitos:** A atividade de levantamento de requisitos (também conhecida como elicitação de requisitos) corresponde à etapa de compreensão do problema aplicada ao desenvolvimento de software.

2. **Análise:** As fases de levantamento de requisitos e de análise de requisitos recebem o nome de engenharia de requisitos. Formalmente, o termo análise corresponde a "quebrar" um sistema em seus componentes e estudar como eles interagem entre si com o objetivo de entender como esse sistema funciona.

3. **Projeto:** O foco principal da análise são os aspectos lógicos e independentes de implementação de um sistema (i.e., os requisitos desse sistema). Na fase de projeto, determina-se "como" o sistema funcionará para atender aos requisitos, de acordo com os recursos tecnológicos existentes.

4. Implementação: o sistema é codificado.

5. **Testes:** Diversas atividades de teste são realizadas para verificação do sistema construído, levando-se em conta a especificação feita nas fases de análise e de projeto.

6. **Implantação:** Na fase de implantação, o sistema é empacotado, distribuído e instalado no ambiente do usuário.

1.1.3 Modelo de processo de software

Para Sommerville 2003 um modelo de processo de software é uma descrição simplificada de um processo de software, que é apresentada a partir de uma perspectiva específica. Ou seja, é uma abstração do processo real que está sendo descrito. Existe uma série de diferentes modelos gerais, ou paradigmas, de desenvolvimento de software:

1.1.3.1 Cascata

Este ciclo de vida também é chamado de clássico ou linear e se caracteriza por possuir uma tendência na progressão sequencial entre uma fase e a seguinte. Eventualmente, pode haver uma retroalimentação de uma fase para a anterior, mas, de um ponto de vista macro, as fases seguem sequencialmente [Bezerra 2015]

Esse modelo considera as atividades de especificação, desenvolvimento, validação e evolução, que são fundamentais ao processo, e as representa como fases separadas do processo, como a especificação de requisitos, o projeto de software, a implementação, os testes e assim por diante.

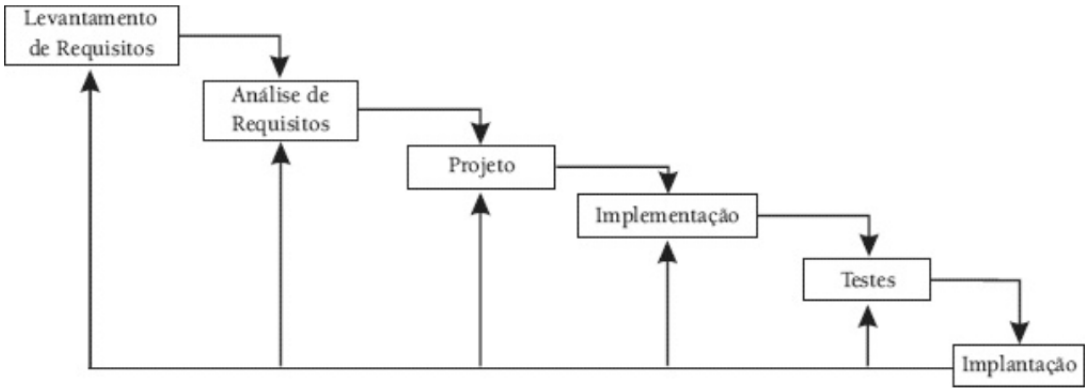

Figura 1 – Modelo cascata [Bezerra 2015]

1.1.3.2 O modelo de processo de software iterativo e incremental

Segundo Bezerra 2015 , o modelo de processo de software incremental e iterativo foi proposto como uma resposta aos problemas encontrados no modelo em cascata. Um processo de desenvolvimento utilizando essa abordagem divide o desenvolvimento de um produto de software em ciclos. Em cada ciclo dessa etapa podem ser identificadas as fases de análise, projeto, implementação e testes. Essa característica contrasta com a abordagem clássica, na qual as fases de análise, projeto, implementação e testes são realizadas uma única vez.

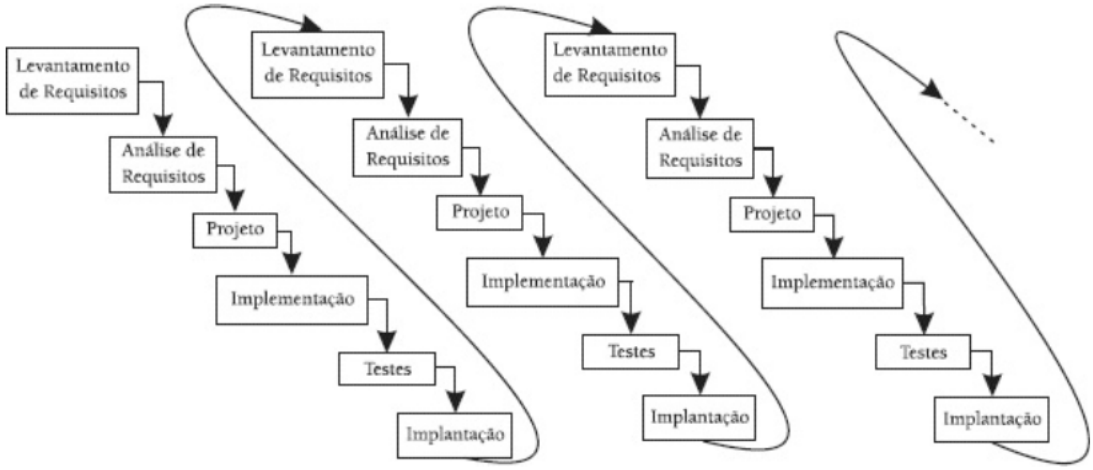

Figura 2 – No processo incremental e iterativo, cada iteração é uma minicascata. [Bezerra 2015]

1.1.3.3 Modelos de processo evolucionário

Software, assim como todos sistemas complexos, evolui ao longo do tempo. Conforme o de- senvolvimento do projeto avança, as necessidades de negócio e de produto mudam frequente- mente, tornando inadequado seguir um planejamento em linha reta de um produto final [Pressman e Maxim 2016]. Assim, faz-se necessário um modelo de processo que tenha sido projetado especificamente para desenvolver um produto que evolua ao longo do tempo.

Modelos evolucionários são iterativos. Apresentam características que possibilitam desen- volver versões cada vez mais completas do software. Nos parágrafos seguintes, são apresenta- dos dois modelos comuns em processos evolucionários [Pressman e Maxim 2016].

1.1.3.3.1 Prototipação

Na sua forma ideal, o protótipo atua como um mecanismo para identificar os requisitos do software. Caso seja necessário desenvolver um protótipo operacional, pode-se utilizar partes de programas existentes ou aplicar ferramentas (por exemplo, geradores de relatórios e gerencia- dores de janelas) que possibilitem gerar rapidamente tais programas operacionais.

O protótipo pode servir como “o primeiro sistema”. Aquele que Brooks recomenda que se jogue fora. Porém, essa pode ser uma visão idealizada. Embora alguns protótipos sejam cons- truídos como “descartáveis”, outros são evolucionários, no sentido de que evoluem lentamente até se transformar no sistema real [Pressman e Maxim 2016].

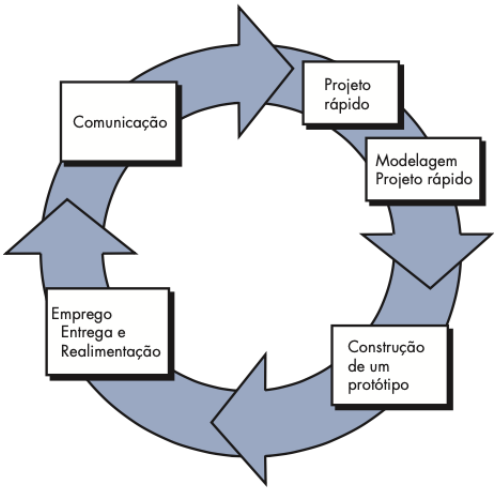

Figura 3 – Modelo Prototipação [Pressman e Maxim 2016]

1.1.3.3.2 Modelo Espiral

É um modelo de processo de software evolucionário que acopla a natureza iterativa da prototi- pação com os aspectos sistemáticos e controlados do modelo cascata. Fornece potencial para o rápido desenvolvimento de versões cada vez mais completas do software [Pressman e Maxim 2016].

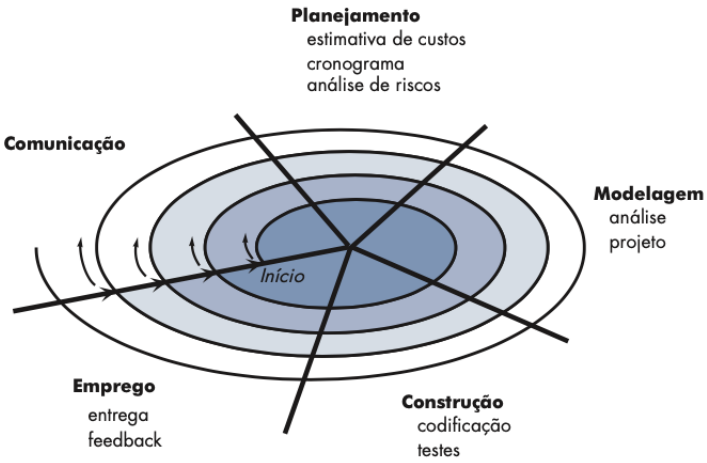

Figura 4 – Modelo em espiral [Pressman e Maxim 2016]

O modelo espiral é uma abordagem realista para o desenvolvimento de sistemas e de software em larga escala. Pelo fato de o software evoluir à medida que o processo avança, o desenvolvedor e o cliente compreendem e reagem melhor aos riscos[1] em cada nível evolucionário.

[1] Risco pode ser entendido como a avaliação da probabilidade de um perigo ocorrer e no cálculo de seu possível impacto e prejuízo para a corporação.

2 Engenharia de requisitos

Um ladrão rouba um tesouro, mas não furta a inteligência.
Uma crise destrói um herança, mas não uma profissão.
Não importa se você não tem dinheiro, você é uma pessoa rica,
pois possui o maior de todos os capitais: a sua inteligência.
Invista nela. ***Estude!.***
Augusto Cury

Os problemas que os engenheiros de software têm para solucionar são, muitas vezes, imensamente complexos. Compreender a natureza dos problemas pode ser muito difícil, especial- mente se o sistema for novo. Consequentemente, é difícil estabelecer com exatidão o que o sistema deve fazer. As descrições das funções e das restrições são os requisitos para o sistema; e o processo de descobrir, analisar, documentar e verificar essas funções e restrições é chamado de **engenharia de requisitos** [Sommerville 2003].

Para [Sommerville 2003], o termo requisito é usado de forma errada pela industria. Assim, o autor separa a ideia de requisitos em três grupos, os quais são:

1. **Requisitos do usuário** são declarações, em linguagem natural e também em diagramas, sobre as funções que o sistema deve fornecer e as restrições sob as quais deve operar.

2. **Requisitos de sistema** estabelecem detalhadamente as funções e as restrições de sistema. O documento de requisitos de sistema, algumas vezes chamado de especificação funcional, deve ser preciso. Ele pode servir como um contrato entre o comprador do sistema e o desenvolvedor do software.

Diferentes níveis de especificação de sistema são úteis porque comunicam informações sobre o sistema para diferentes tipos de leitores. A Figura 5 ilustra a distinção entre os requisitos de usuários e os de sistemas. Ela mostra como um requisito de usuário pode ser expandido em diversos requisitos de sistema.

Definição dos requisitos do usuário

1. O software deve oferecer um meio de representar e acessar arquivos externos criados por outras ferramentas.

Especificação dos requisitos de sistema

1.1 O usuário deve dispor de recursos para definir o tipo dos arquivos externos.
1.2 Cada tipo de arquivo externo pode ter uma ferramenta associada que pode ser aplicada a ele.
1.3 Cada tipo de arquivo externo pode ser representado como um ícone específico na tela do usuário.
1.4 Devem ser fornecidos recursos para o ícone que representa um arquivo externo, a ser definido pelo usuário.
1.5 Quando um usuário seleciona um ícone que representa um arquivo externo, o efeito dessa seleção é aplicar a ferramenta associada com o tipo de arquivo externo ao arquivo representado pelo ícone selecionado

Figura 5 – Requisitos do usuário e do sistema. [Sommerville 2003]

2.1 Requisitos funcionais e não funcionais

Os requisitos de sistema de software são, frequentemente, classificados como funcionais ou não funcionais:

1. **Requisitos funcionais:** São declarações de funções que o sistema deve fornecer, como o sistema deve reagir a entradas específicas e como deve se comportar em determinadas situações. Em alguns casos, os requisitos funcionais podem também explicitamente declarar o que o sistema não deve fazer.

2. **Requisitos não funcionais:** São restrições sobre os serviços ou as funções oferecidos pelo sistema. Entre eles destacam-se restrições de tempo, restrições sobre o processo de desenvolvimento, padrões, entre outros.

Referências

BEZERRA, E. *Princípios de Análise e Projeto de Sistemas com UML*. [S.l.]: Elsevier Rio de Janeiro, 2015. v. 3. Citado 2 vezes nas páginas 4 e 5.

PRESSMAN, R.; MAXIM, B. *Engenharia de Software-8ª Edição*. [S.l.]: McGraw Hill Brasil, 2016. Citado 2 vezes nas páginas 5 e 6.

RUIZ, E. E. S. *IBm1030 Programação Orientada a Objetos*. 2008. Disponível em: <http://dcm.ffclrp.usp.br/~evandro/ibm1030/constru/heranca.html>. Nenhuma citação no texto.

SILVA, D. Lucas da et al. Ontologias e unified modeling language: uma abordagem para representação de domínios de conhecimento. v. 10, 10 2009. Nenhuma citação no texto.

SOMMERVILLE, I. *Engenharia de Software. 6ª*. [S.l.: s.n.], 2003. Citado 5 vezes nas páginas 2, 3, 4, 7 e 8.